

```

lubuntu@lubuntu:~/Projects/make-sense-of-containers$ ps
  PID TTY          TIME CMD
 1999 pts/0      00:00:00 bash
 2837 pts/0      00:00:00 ps
lubuntu@lubuntu:~/Projects/make-sense-of-containers$ go run main.go run bash
Running [bash]
lubuntu@lubuntu:~/Projects/make-sense-of-containers$ ps
  PID TTY          TIME CMD
 1999 pts/0      00:00:00 bash
 2838 pts/0      00:00:00 go
 2871 pts/0      00:00:00 main
 2875 pts/0      00:00:00 bash
 2883 pts/0      00:00:00 ps
lubuntu@lubuntu:~/Projects/make-sense-of-containers$

```

Figure 2: Observing the output of `ps` in the host and from within our container

```

root@lubuntu: /home/lubuntu/Projects/make-sense-of-containers# ps
  PID TTY          TIME CMD
 2930 pts/0      00:00:00 sudo
 2931 pts/0      00:00:00 bash
 3048 pts/0      00:00:00 ps
root@lubuntu: /home/lubuntu/Projects/make-sense-of-containers# hostname
lubuntu
root@lubuntu: /home/lubuntu/Projects/make-sense-of-containers# go run main.go run bash
Running [bash]
root@lubuntu: /home/lubuntu/Projects/make-sense-of-containers# ps
  PID TTY          TIME CMD
 2930 pts/0      00:00:00 sudo
 2931 pts/0      00:00:00 bash
 3050 pts/0      00:00:00 go
 3085 pts/0      00:00:00 main
 3090 pts/0      00:00:00 bash
 3097 pts/0      00:00:00 ps
root@lubuntu: /home/lubuntu/Projects/make-sense-of-containers# hostname
lubuntu
root@lubuntu: /home/lubuntu/Projects/make-sense-of-containers# hostname pratikms
root@lubuntu: /home/lubuntu/Projects/make-sense-of-containers# hostname
pratikms
root@lubuntu: /home/lubuntu/Projects/make-sense-of-containers# exit
exit
root@lubuntu: /home/lubuntu/Projects/make-sense-of-containers# hostname
pratikms
root@lubuntu: /home/lubuntu/Projects/make-sense-of-containers#

```

Figure 3: Attempting to change hostname from the container and verifying it later in the host

Namespaces

Namespaces provide the isolation needed to run multiple containers on one machine while giving each what appears like its own environment. There are six namespaces, and each can be independently requested.

PID: The PID namespace gives a process and its children their own view of a subset of the processes in the system. This is analogous to a mapping table. When a process of a PID namespace asks the kernel for a list of processes, the kernel looks at the mapping table. If the process exists in the table, the mapped ID is used instead of the real ID. If it doesn't exist in the mapping table, the kernel pretends it doesn't exist at all. The PID namespace makes the first process created within it PID 1 (by mapping whatever its host ID is to 1), giving the appearance of an isolated process tree in the container. This is a really interesting concept.

MNT: In a way, this one is the most important. The mount namespace gives the processes contained within it their own mount table. This means they can mount and unmount directories without affecting other namespaces including the host namespace. More importantly, in combination with the `pivot_root` syscall, it allows a process to have its own file system. This is how we can have a process think it's running on Ubuntu, CentOS, Alpine, etc — by swapping out the file system that the container sees.

NET: The network namespace gives the processes that use it their own network stack. In general, only the main network namespace (the one that the processes start when you start your computer use) will have any real physical network cards attached. But we can create virtual Ethernet pairs — linked Ethernet cards where one end can be placed in one network namespace and one in another, creating

a virtual link between the network namespaces. This is like having multiple IP stacks talking to each other on one host. With a bit of routing magic, this allows each container to talk to the real world while isolating each to its own network stack.

UTS: The UTS namespace gives its processes their own view of the system's hostname and domain name. After entering a UTS namespace, setting the hostname or the domain name will not affect other processes.

IPS: The IPC namespace isolates various inter-process communication mechanisms such as message queues. This particular namespace deserves a blog post of its own. There's so much to IPC than what I can comprehend myself. This is why I will encourage you to check out the namespace docs (<https://www.man7.org/linux/man-pages/man7/namespaces.7.html>) for more details.

USER: The user namespace was the most recently added and is likely the most powerful from a security perspective. This namespace maps the UIDs to different sets of UIDs (and GIDs) on the host, which is extremely useful. Using a user namespace we can map the container's root user ID (i.e., 0) to an arbitrary and unprivileged UID on the host. This means we can let a container think it has root access without actually giving it any privileges in the root namespace. The container is free to run processes as `uid 0` — which normally would be synonymous with having root permissions, but the kernel is actually mapping that UID under the covers to an unprivileged real UID belonging to the host OS.

Most container technologies place a user's process into all of the above namespaces and initialise them to provide a standard environment. This amounts to, for example, creating an initial Internet card in the isolated network namespace of the container, with connectivity to a real network on the host. In our case, for satisfying our immediate requirement, we will add the UTS namespace to our script